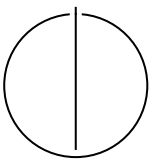
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Computational Science and Engineering

Fast Multipole Boundary Element Method for Finite Periodic Structures

Abhijeet Abhay Sutar



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

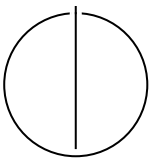
TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Computational Science and Engineering

Fast Multipole Boundary Element Method for Finite Periodic Structures

Titel der Abschlussarbeit

Author:	Abhijeet Abhay Sutar
Examiner:	Steffen Marburg, Prof. Dr.
Supervisor:	Arne Hildenbrand, M.Sc.
Submission Date:	Submission date



I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, Submission date Abhijeet Abhay Sutar

Acknowledgments

Abstract

Contents

1. Introduction

1.1. Section

Citation test [Lam94].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

For more details, see the documentation of the `acronym` package[1].

1.1.1. Subsection

See Table 1.1, Figure 1.1, Figure 1.2, Figure 1.3.

Table 1.1.: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

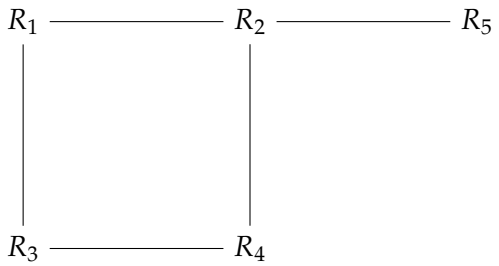


Figure 1.1.: An example for a simple drawing.

[1] `https://ctan.org/pkg/acronym`

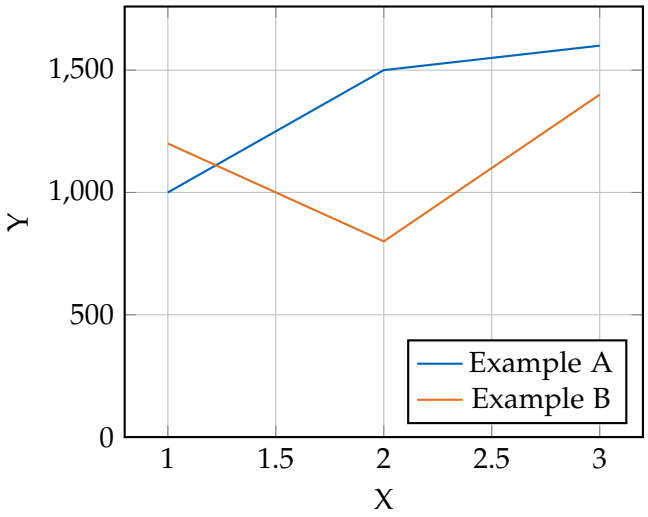


Figure 1.2.: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 1.3.: An example for a source code listing.

2. Mathematical Formulation of BEM

Before we discus the implementation of the Boundary Element Method (BEM) for the Acoustic Helmholtz equation, we first need

2.1. Acoustic Wave Equation

The propagation of acoustic waves in a homogeneous, inviscid fluid medium is governed by the linear wave equation for the acoustic pressure $p(\mathbf{x}, t)$:

$$\nabla^2 p(\mathbf{x}, t) - \frac{1}{c^2} \frac{\partial^2 p(\mathbf{x}, t)}{\partial t^2} = 0 \tag{2.1}$$

where $\mathbf{x}$ is the position vector, t is time, and c is the speed of sound in the medium.

To derive the Helmholtz equation, we consider a time-harmonic acoustic field. We assume the pressure can be represented as a complex function oscillating with angular frequency ω:

$$p(\mathbf{x}, t) = P(\mathbf{x}) e^{-i\omega t} \tag{2.2}$$

where $P(\mathbf{x})$ is the spatial component of the pressure (the phasor) and $i = \sqrt{-1}$ is the imaginary unit.

Substituting Equation (2.2) into the wave equation (2.1):

$$\nabla^2 \left(P(\mathbf{x}) e^{-i\omega t} \right) - \frac{1}{c^2} \frac{\partial^2}{\partial t^2} \left(P(\mathbf{x}) e^{-i\omega t} \right) = 0 \tag{2.3}$$

Calculating the time derivatives:

$$\frac{\partial}{\partial t} \left(P(\mathbf{x}) e^{-i\omega t} \right) = -i\omega P(\mathbf{x}) e^{-i\omega t} \tag{2.4}$$

$$\frac{\partial^2}{\partial t^2} \left(P(\mathbf{x}) e^{-i\omega t} \right) = (-i\omega)^2 P(\mathbf{x}) e^{-i\omega t} = -\omega^2 P(\mathbf{x}) e^{-i\omega t} \tag{2.5}$$

Substituting this back into the wave equation:

$$e^{-i\omega t} \nabla^2 P(\mathbf{x}) - \frac{1}{c^2} (-\omega^2 P(\mathbf{x}) e^{-i\omega t}) = 0 \tag{2.6}$$

Dividing by the common time factor $e^{-i\omega t}$ (which is non-zero) and rearranging the terms:

$$\nabla^2 P(\mathbf{x}) + \frac{\omega^2}{c^2} P(\mathbf{x}) = 0 \tag{2.7}$$

Introducing the wavenumber $k = \omega/c$, we arrive at the homogeneous Helmholtz equation:

$$\nabla^2 P(\mathbf{x}) + k^2 P(\mathbf{x}) = 0 \tag{2.8}$$

2.2. Boundary Element Method

2.2.1. Boundary Integral Formulation

The Boundary Element Method (BEM) solves the Helmholtz equation by transforming it into an integral equation defined on the boundary of the domain. This transformation is typically achieved using Green's Second Identity.

Let Ω be the domain containing the fluid and Γ be its boundary. For two scalar fields P and G in Ω, Green's Second Identity states [Wu00]:

$$\int_\Omega \left(P\nabla^2 G - G\nabla^2 P\right) d\Omega = \int_\Gamma \left(P\frac{\partial G}{\partial n} - G\frac{\partial P}{\partial n}\right) d\Gamma \tag{2.9}$$

where n is the outward unit normal vector to the boundary.

We choose $P(\mathbf{x})$ to be the acoustic pressure, satisfying the Helmholtz equation derived in Equation (2.8):

$$\nabla^2 P(\mathbf{x}) + k^2 P(\mathbf{x}) = 0 \tag{2.10}$$

We choose $G(\mathbf{x}, \mathbf{y})$ to be the fundamental solution (Green's function) of the Helmholtz equation, which satisfies the inhomogeneous equation with a point source at $\mathbf{y}$:

$$\nabla^2 G(\mathbf{x}, \mathbf{y}) + k^2 G(\mathbf{x}, \mathbf{y}) = -\delta(\mathbf{x} - \mathbf{y}) \tag{2.11}$$

where δ is the Dirac delta function. For 3D acoustics, the free-space Green's function satisfying the Sommerfeld radiation condition is given by [Kir98]:

$$G(\mathbf{x}, \mathbf{y}) = \frac{e^{ikr}}{4\pi r} \quad \text{with} \quad r = |\mathbf{x} - \mathbf{y}| \tag{2.12}$$

Substituting these into Green's Second Identity for a source point $\mathbf{y}$ inside the domain Ω:

$$P(\mathbf{y}) = \int_{\Gamma} \left(G(\mathbf{x},\mathbf{y}) \frac{\partial P}{\partial n}(\mathbf{x}) - P(\mathbf{x}) \frac{\partial G}{\partial n}(\mathbf{x},\mathbf{y}) \right) d\Gamma(\mathbf{x}) \tag{2.13}$$

As the source point $\mathbf{y}$ is moved to the boundary Γ, the integrals become singular. Evaluating the singularity using a limiting process yields the Boundary Integral Equation (BIE) [Mar08]:

$$C(\mathbf{y})P(\mathbf{y}) = \int_{\Gamma} \left(G(\mathbf{x},\mathbf{y}) \frac{\partial P}{\partial n}(\mathbf{x}) - P(\mathbf{x}) \frac{\partial G}{\partial n}(\mathbf{x},\mathbf{y}) \right) d\Gamma(\mathbf{x}) \tag{2.14}$$

where $C(\mathbf{y})$ is the geometric coefficient (solid angle). For a smooth boundary, $C(\mathbf{y}) = 0.5$.

2.3. Discretization

To solve the BIE numerically, the boundary Γ is discretized into N boundary elements. The pressure P and its normal derivative $q = \partial P/\partial n$ are approximated using shape functions.

Applying the collocation method at the nodes leads to a system of linear algebraic equations:

$$\mathbf{HP} = \mathbf{GQ} \tag{2.15}$$

where $\mathbf{H}$ and $\mathbf{G}$ are the dense, nonsymmetric influence matrices resulting from the integration of the fundamental solution and its derivative, and $\mathbf{P}$ and $\mathbf{Q}$ are vectors containing the nodal values of pressure and normal velocity, respectively.

2.4. Burton-Miller Formulation

The Boundary Element Method (BEM) offers a highly elegant, mathematically rigorous, and computationally efficient alternative by reformulating the volumetric partial differential equation into a Boundary Integral Equation (BIE) mapped strictly over the surface of the scattering or radiating body. This approach intrinsically satisfies the Sommerfeld radiation condition at infinity, thereby reducing the dimensionality of the problem by one—transforming a three-dimensional volumetric domain into a two-dimensional surface mesh, or a two-dimensional problem into a one-dimensional contour [Lam94; Wu00; Mar08; Kir98].

Remove test citation

However, as [BM71] showed, the standard BEM approach for exterior problems, gives rise to non-existence of unique solutions at wavenumbers coinciding with the resonances of the corresponding interior problem.

At these critical frequencies, also sometime referred to as *fictitious or irregular frequencies*, the matrix system resulting from the discretization of the BIE becomes ill-conditioned, leading to numerical instability and a spike in the error of the computed solution.

It must be noted that the original exterior problem does have a unique solution at these frequencies and that the connection to the interior problem has no physical significance. The non-uniqueness arises solely from the formulation of the problem in terms of a Boundary Integral Equation (BIE).[BM71]

3. The Fast Multipole Method Framework

The BEM lends itself naturally to modelling acoustic wave propagation. Transforming the volumetric governing differential equations into boundary integral equations allows finite structures to be treated in an infinite exterior domain without requiring special boundary conditions. However, the required mathematical transformation of the problem yields a full, complex, non-Hermitian matrix [CL07]. As the degrees of freedom increase to model larger problem, the memory requirements to hold the large full matrix and the cost to compute the matrix coefficients themselves, scale as $\mathcal{O}(N^2)$, and the computational cost of solving the resulting linear system using direct solvers scales as $\mathcal{O}(N^3)$, where N is the number of degrees of freedom [Liu09]. This makes the BEM computationally prohibitive for large-scale problems.

better ending for that line please.

To overcome this computational bottleneck, we must employ a radically different numerical strategy. This chapter details the algorithmic design and the mathematical formulation of the FMM framework. The FMM decouples far-field interactions through hierarchical spatial decomposition, truncated analytical basis expansions, and the application of complex mathematical translation operators [CRW93].

This chapter focuses on the mathematical and algorithmic design of the FMM implemented as a part of this thesis. It details the division of the mesh space into a topological tree data structure, the mathematics of the multi-stage multipole translations, and the implementation of near-field interactions.

3.1. Fast Multipole Method

The FMM was introduced in 1987 by Greengard and Rokhlin [GR87] to accelerate large-scale particle simulations involving long-range pairwise potential interactions. Later, it was extended to other problems, including the BEM for Acoustic Scattering [Rok90]. The FMM handles short-range interactions with direct computation, and uses analytical multipole expansions to compute potentials for long-range interactions. By providing a method for the rapid computation of the matrix-vector product between the system matrix any arbitrary vector, the FMM enables the use of iterative solvers for solving the linear systems arising from the BEM discretization, reducing the computational complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N^{\frac{3}{2}})$ or even $\mathcal{O}(N \log N)$ for a multilevel implementation [CRW93].

3.1.1. The FMM algorithm

The FMM follows these basic steps:

Step 1. Discretization Any problem begins with discretizing the boundary, as is done for conventional BEM. We use constant elements.

Step 2. Initializing the Octree For a 3D problem, we consider a cube that contains the entire geometry. This cube is repeatedly subdivided into 8 smaller cubes, called its child nodes, creating an Octree structure. The subdivision continues until the number of elements in a cube falls below a defined threshold.

Step 3. Upward Pass Beginning from the bottom of the tree, at the deepest level called the leaf nodes, we compute the moments of each node, tracing the tree structure upwards till we are two levels below the root node. Using the Multipole-to-Multipole (M2M) translation operator, the moments of the child nodes are translated and aggregated to the parent node.

Step 4. Downward Pass Starting from the second level of the tree, we compute the local expansions for each node. This consists of contributions from the current node's interaction list and from far-off nodes. The interaction list nodes are those not directly adjacent to the current node, but whose parent node is adjacent to the current node's parent. The contribution from the interaction list nodes is computed using the M2L translation operator, while the contribution from the far-off nodes is computed using the L2L translation operator for the parent node, with the expansion point being shifted from the centroid of the parent node to the centroid of the current node. At level 2, the nodes only have contributions from the interaction list nodes. Since there are no far-off nodes at this level, we use the M2L operator to get the local expansion's coefficients.

Step 5. Evaluating of the Integrals The contributions of all the collocation points in the current node and thed nodes directly adjacent to it are computed using direct BEM. The contributions of all nodes in the interaction list and the far-off are computed using the local expansion coefficients of the current node and shifting the expansion point to the collocation point using the Local-to-Particle (L2P) translation operator.

Step 6. Iterative Solver We update the unknown vector, in this case the `phi` vector in the system of equations $\mathbf{A}\boldsymbol{\phi} = \mathbf{b}$, using an iterative solver, such as GMRES, and repeat from steps 3 until convergence is achieved.

3.2. Octree

The spatial domain can be subdivided into a hierarchical tree structure in many ways. For 3D problems, the most common approach is to use an Octree, which offers many opportunities to optimize algorithmic efficiency. An Octree is a tree data structure where each node is bisected along the Cartesian x, y, and z axes, producing 8 equally sized child nodes, and these child nodes are then further subdivided into 8 nodes until a certain threshold is reached.

3.2.1. Nodes of the Octree

We define an `FMMNode` data structure to represent each node in the Octree, which contains the following information:

Node geometry: The struct keeps track of the node's bounding box, its center, and its radius. It also stores the integer grid coordinates that represent the node's position in the Octree.

Tree Hierarchy: Each node is assigned a node index, which serves as the node's unique identifier within a global list of all the nodes. The node also stores its tree level and the index of its parent node. Most importantly, it stores the indices of its 8 child nodes, which are used to navigate the tree.

Mesh and Interaction data: The node stores an iterator for the particles contained within it. It maintains a neighbor list that contains the indices of neighboring nodes. These interactions must be computed directly. It also stores a list of far-field nodes, which are sufficiently far away to be approximated by multipole expansions.

Mathematical expansion data: The node stores `Pm` which is the order used for the mutlitpole expansion, and the number of coefficients required based on the order. It also stores the multipole and local expansion coefficients, which are used to approximate the potential and forces from distant nodes.

FMM Translation Operators: The node stores the necessary translation operators for the FMM algorithm.

1. Particle-to-Multipole (P2M) Matrix: Translates raw particles (mesh sources) to a node's Multipole expansion.
2. M2M Matrix: Translates a child's Multipole expansion up to its parent's Multipole expansion.

3. Multipole-to-Local (M2L) Matrices: Converts a distant node's Multipole expansion into this node's Local expansion.
4. Local-to-Local (L2L) Matrix: Translates a parent's Local expansion down to its child's Local expansion.
5. L2P Matrix: Evaluates the Local expansion directly onto the target particles (mesh cells).

add a figure of Octree

3.2.2. Splitting the domain to make an Octree

The underlying spatial data-structue sets the stage for the computational efficiency of the FMM algorithm. It dictates how the source-target interactions are categorized as either near-field, needing direct computation, or far-field, and thus suitable for approximation via multipole expansions. An adaptive, heirarchical Octree allows for a systematic way to manage these interactions.

Space-Filling Curves and Morton Encoding

We map the geometric centers of the boundary elements from three-dimensional space into a one-dimensional array using a space-filling curve, specifically, the Morton Z-curve. Morton encoding converts the 3D coordinates of each mesh element into a single integer by interleaving the bits of the binary representations of the x, y, and z coordinates, generating that element's Morton key. This ordering assigns numerically adjacent Morton keys to spatially adjacent elements. This allows us to sort the elements into a linearly ordered array, where adjacent elements get clustered together, improving cache locality. This cache locality helps reduce the overhead of assigning the elements to the Octree's nodes. While the Morton Z-order curve may not be the optimum space-filling curve for 3D problems, its simplicity of implementation via bitwise operations lends itself to efficient execution [MB15].

4. Software Architecture and HPC Optimizations

4.1. Bitwise Operations for Morton Encoding

As detailed in Section 3.2.2, the Morton Z-order curve is used to map the 3D coordinates of the mesh elements for a more efficient Octree construction. To optimize this process without relying on expensive looping, we use a series of bitwise operations that interleave the bits of the binary representations of the x, y, and z coordinates.

The basic algorithm that we use to compute the Morton key for a given 3D coordinate is as follows:

Step 1. Grid Quantization: We define a discrete grid across the global bounding box. We define a discrete grid across the global bounding box. With a 64-bit integer, we can divide each spatial dimension into 2^{20} $(1,048,576)$ bins, which allows us to represent up to 2^{60} unique spatial locations, more than enough for our problem size.

Step 2. Coordinate Normalization: We normalize the x, y, and z coordinates of each mesh element to fit within $[0,1]$ relative to the global bounding box, and then multiply by the number of bins to get the quantized integer coordinates.

Step 3. Bit Truncation: We ensure that the quantized coordinates fit within 21 bits, ensuring that they can be interleaved into a single 64-bit integer without overflow. This is achieved by applying a bitwise `AND` operation with a mask that retains only the lower 21 bits of each coordinate.

Step 4. Bit Expansion: We use a sequence of bitwise shifts and bitwise operations with bit masks to add two zero bits between each bit of the quantized coordinates. This process is repeated for each coordinate.

Step 5. Bit Interleaving: Finally, we combine the expanded bits of each coordinate by shifting the expanded y bits by one, and the expanded z bits by two, and then performing a bitwise OR operation to interleave the bits together, resulting in the final Morton key for that mesh element.

The basic algorithm of this process is shown for the 2D case in [And05]. The exact implementation for the 3D case is shown in the appendix A.1

4.2. Caching the Matrices

The FMM requires us to compute a lot of translations matrices, such as the P2M, M2M, M2L, L2L, and L2P matrices. The computation of all these matrices, and especially the M2L matrices, is highly compute-bound. This stems from the fact that calculating the M2L operators requires evaluating the spherical harmonics, associated Legendre polynomials, Wigner 3-j symbols, and the Gaunt coefficients, which make heavy use of recursive branching and high-latency floating point division. This causes both, low arithmetic intenstity and severe pipeline stalls, representing a significant bottleneck in the FMM algorithm.

We mitigate this by exploiting the strict geometric symmetry of our uniform Octree. If we consider the geometry of the Octree, we can see that at a particular level, the boundaries of the interaction domain form a localized $7 \times 7 \times 7$ grid of uniform boxes around the target node, as shown in Figure **??**. Discounting the center box, which is the target node itself, and the 26 boxes that are direct neighbors of the target node, which are handled by direct computation, we are left with $7^3 - 1 - 26 = 316$ possible M2L translation operators that we need to compute per tree level. [Koh+21]

Since the number of unique M2L operators is limited to 318 per tree level, this makes it practically feasible for us to precompute and store these matrices. We further optimize this by generating a lookup table that maps each M2L matrix to a transfer ID. To index this lookup table, we rely on the fact that with our $7 \times 7 \times 7$ interaction grid, the relative coordinate offsets in $3D$ space is strictly bounded by the integer range $[-3, 3]$ in each dimension. By shifting this coordinate range by $+3$, we can build our transfer ID Like so:

$$\textbf{Transfer ID} = (\delta x + 3) + 7\,((\delta y + 3) * 7 + (\delta z + 3))$$

While we allocate memory for all 342 possible M2L matrices, we only compute and store the 316 matrices that correspond to the interaction list. This allows us to seamlessly use the transfer ID as an index into the lookup table, without needing costly floating point comparisons to check if a particular M2L interaction is valid or not. This optimization allows us to skip the extremely expensive computation of the M2L matrices, turning the requirement of performing complex mathematical operations into a trivial $\mathcal{O}(1)$ lookup.

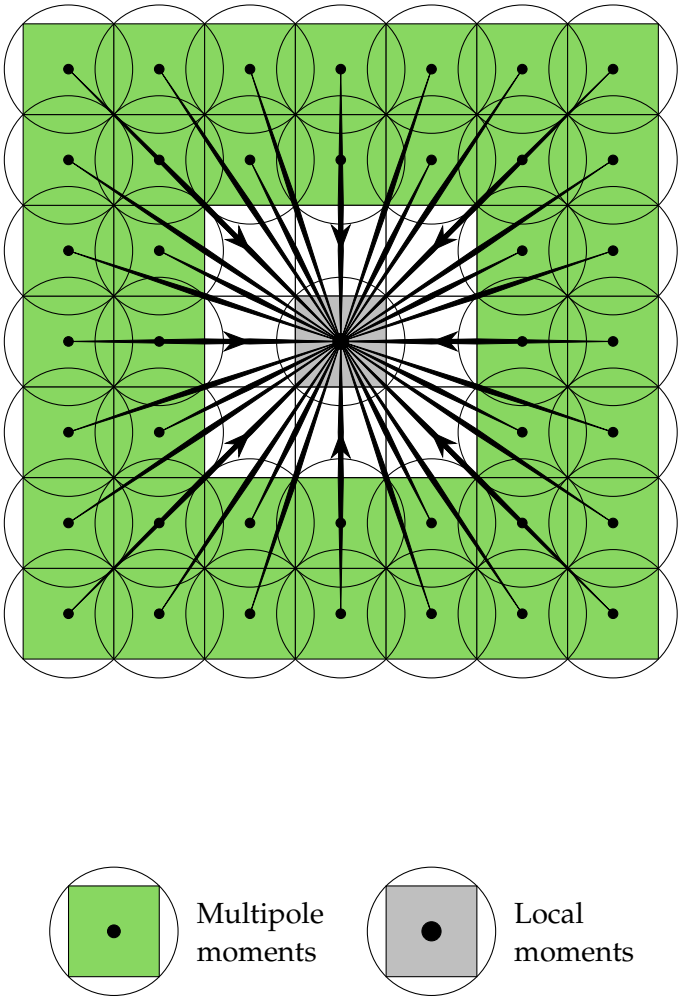


Figure 4.1.: A 2D representation of the M2L interaction list. In 3D, there are $7^3 - 1 = 342$ possible relative positions for the M2L interactions.
[Koh+21]

4.3. OpenMP Speed Up

Modern Central Processing Unit (CPU)s are equipped with multiple cores and can execute multiple threads in parallel. On top of that, they support Single Instruction Multiple Data (SIMD) operations, which allows them to perform the same operation on multiple data points simultaneously. The OpenMP Application Programming Interface (API) provides a simple and effective way to parallelize code to take advantage of these features. We leverage OpenMP by setting `#pragma` directives in the region of our code where we want parallelism. These directives instruct the compiler to generate code that can be executed in parallel across multiple threads. OpenMP allows us to control parallelism at both the coarse-grained thread-level and the fine-grained CPU instruction level. The particular strategies we have used in the code are as follows:

4.3.1. Loop Collapsing

Building the global M2L cache as detailed in the previous section helps us bypass particularly troublesome bottleneck. However, there is still more performance to be wrung out of the M2L phase. Computing the global M2L cache requires iterating over the $7 \times 7 \times 7$ grid of possible interactions for each level. This uses three nested loops, which when naïvely parallelized with the standard `#pragma omp parallel for` directive, would only result in a maximum parallel iteration space of 7. Modern CPUs with many more threads available would end up being underutilized. To solve this, and maximize the workload distribution across all cores, we use the `#pragma omp parallel for collapse(3)` directive.

The `collapse(3)` clause tells the compiler to collapse the three hierarchically nested loops into a single loop with a larger iteration space of $7 \times 7 \times 7 = 343$. Each thread then gets assigned a chunk of this larger iteration space, allowing us to fully utilize the available threads and achieve better load balancing.

4.3.2. Dynamic Scheduling

While a highly structured, uniform iteration workload would fit perfectly with OpenMP's default static scheduling, the FMM Octree's workload is inherently irregular. The number of mesh elements in each node can vary widely, especially higher up in the tree where some nodes have stopped subdividing due to a lack of elements. Also, the size of each node's interaction list varies, depending on its spatial location. This leads to a highly imbalanced workload across the nodes, which can cause significant load imbalance when naïvely calling the `#pragma omp parallel for` directive on the Octree traversal loops. The default static scheduling, when called over the `total_nodes`, would

divide the total number of nodes by the number of worker threads, would result in considerable load imbalance.

Using the `#pragma omp parallel for schedule(dynamic)` directive is the perfect counter to this. Dynamic scheduling instructs the OpenMP runtime to initialize a centralized work queue, and as worker threads become idle, they pull the next available chunk of iterations from the queue. This dynamic load balancing ensures that no thread sits idle while the work queue is not empty [Klo20].

4.3.3. SIMD Vectorization

Leveraging thread-level parallelism with OpenMP is a great way to speed up the FMM algorithm, but we can take this one step further with instruction-level parallelism. Modern CPUs are equipped with SIMD vector units that enable them to perform the same operation on multiple data points simultaneously [XWZ23]. Instruction sets like the AVX-256 or AVX-512 (Advanced Vector Extensions) use large hardware registers, in the case of the AVX-512, 512 bits wide, which can hold four `std::complex<double>`(64 bits each for the real and imaginary parts) values at once. OpenMP's `#pragma omp simd` allows us to instruct the compiler to generate these specific hardware-level instructions needed to take advantage of these vector units. The general rule for where we can apply this directive is to look for innermost loops that perform simple, predictable arithmetic (especially reductions like `sum += ...`) without branching (`if/else`), function calls, or memory allocations. We use this directive in the innermost loops of the `compute_upward_pass`, `compute_horizontal_pass`, and `compute_downward_pass` functions, which perform the core arithmetic of the FMM algorithm. All these inner loops are of the type `sum += A[i] * B[i]`. These get compiled to the hardware-level Fused Multiply-Add (FMA) instruction, which performs multiplication and addition in a single step, reducing latency and improving performance [Fog22].

4.4. Solving the system - vmult + GMRES + The Real-Equivalent system

The BEM discretization leads to a dense, complex-valued linear system that is complex-symmetric but not Hermitian, and therefore cannot be solved by methods such as the Conjugate Gradient [SB]. While a direct solver can be used, it scales as $\mathcal{O}(N^3)$, which quickly becomes infeasible for large systems. Iterative solvers offer a great alternative, reducing computational cost to $\mathcal{O}(N_iterN^2)$, especially when combined with the FMM's shift to matrix-free operator evaluations that does not require storing all N^2 matrix elements [GD09].

We chose to use the `deal.II` library [Arn+] to implement our BEM and FMM algorithms, as it provides robust and optimized linear algebra and mesh management infrastructure. The `deal.II` library provides a built-in implementation of the GMRES iterative solver. However, it is only intended for real-valued systems [BC]. This limitation is fundamentally a part of the `deal.II` implementation of its `SolverGMRES` [DII:GMRES].

We get around this limitation by formulating our complex-valued system as its equivalent real-valued system [DH01]. The breakdown of our approach is as follows:

1. **Vector Splitting:** We split our complex vectors like `system_rhs` and `phi`, which are of size N, into real `Vector<double>`s of size $2N$, with the first N elements being their real components, and the next N elements being their imaginary parts.

2. **Matrix-Free Wrapper:** We implement a struct `RealFMMOperator` that inherits from `dealii::Subscriptor` and implements a real-valued `vmult` function that the solver expects. It is important to inherit from `dealii::Subscriptor` to ensure that the internal pointers to the FMM data are properly reference-counted and do not lead to memory leaks or dangling pointers.

3. `vmult` **Translations:** Inside our custom `vmult` method, it takes the $2N$-sized real vector and reconstructs the original size N complex vector. It then calls the original complex-valued `vmult` method of our FMM implementation, which performs the matrix-free matrix-vector multiplication. Finally, it takes the resulting complex vector and splits it back int o size $2N$ real vector to be returned to the iterative solver.

4. **Post-Processing:** After the GMRES solver converges, we take the resulting size $2N$ real solution vector and reconstruct the complex N-sized `phi` vector, which contains the values of the potential at the collocation points.

Our implementation of the `RealFMMOperator` struct is shown in Appendix A.2.

While iterative Krylov space solvers like GMRES are quite robust in their convergence, convergence depends on the matrix's spectral properties and condition number [SS86]. The real-equivalent formulations can, in theory, produce eigenvalue distributions that are unfavorable for the convergence of GMRES. To ensure fast convergence, we employ a preconditioner to improve the system's spectral properties. The Jacobi-block-diagonal preconditioner [DH01] we implement extracts the complex-scalar diagonal from the near-field sparse matrix we assembled for the FMM and applies its inverse to the equivalent real $2N \times 2N$ matrix. The exact implementation of this preconditioner is shown in Appendix A.3.

5. Performance Analysis and Validation

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

6. Conclusion and Future Directions

- Currently Burton - Miller is highly optimized for single dof per cell and for perfectly flat panels. That is, FE_Degree = 0 and Mapping_Degree = 1. To support higher degrees, we need to update the hypersingular calculation in the near field operator and the main assemble system method.
- There is some basic parallelization in the code for computing the translation matrices and for the matrix-vector multiplication done with OpenMP pragmas. But, Deal.ii has its native parallelization support using its native datatypes. The FMM also lends itself to parallelization where each node of the FMM tree may reside in an individual process and maintain its own near field matrix, with only the far-off node dipoles having to be passed around.
- The current implementation of the computation of the external field uses drect computations of the green's function. This can be upgraded to use the FMM approximation in the future, which will be faster for large problems.
- The current implementation is strictly for scattering off of sound hard objects. The next step would be to implment the option to set an arbitrary impedance boundary condition.
- The current implementation is strictly for a stationary mesh.

Abbreviations

BEM Boundary Element Method

FMM Fast Multipole Method

BIE Boundary Integral Equation

M2M Multipole-to-Multipole

M2L Multipole-to-Local

L2L Local-to-Local

L2P Local-to-Particle

P2M Particle-to-Multipole

GMRES Generalized Minimal Residual Method

CPU Central Processing Unit

API Application Programming Interface

SIMD Single Instruction Multiple Data

FMA Fused Multiply-Add

List of Figures

List of Tables

Bibliography

[And05] S. E. Anderson. *Bit Twiddling Hacks*. Bit Twiddling Hacks. May 5, 2005. URL: https://graphics.stanford.edu/~seander/bithacks.html#InterleaveBMN (visited on 03/21/2026).

[Arn+] D. Arndt, W. Bangerth, M. Bergbauer, B. Blais, M. Fehling, R. Gassmöller, T. Heister, L. Heltai, M. Kronbichler, M. Maier, P. Munch, S. Scheuerman, B. Turcksin, S. Uzunbajakau, D. Wells, and M. Wichrowski. "The Deal.II Library, Version 9.7, 2025." In: ().

[BC] W. Bangerth and Y.-Y. Cai. *The Deal.II Library: The Step-58 Tutorial Program*. The deal.II Library. URL: https://dealii.org/current/doxygen/deal.II/step_58.html (visited on 03/24/2026).

[BM71] A. J. Burton and G. F. Miller. "The Application of Integral Equation Methods to the Numerical Solution of Some Exterior Boundary-Value Problems." In: *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences* 323.1553 (June 8, 1971), pp. 201–210. ISSN: 0080-4630, 2053-9169. DOI: 10.1098/rspa.1971.0097.

[CL07] S. Chandler-Wilde and S. Langdon. "Boundary Element Methods for Acoustics." In: (July 18, 2007).

[CRW93] R. Coifman, V. Rokhlin, and S. Wandzura. "The Fast Multipole Method for the Wave Equation: A Pedestrian Prescription." In: *IEEE Antennas and Propagation Magazine* 35.3 (June 1993), pp. 7–12. ISSN: 1045-9243. DOI: 10.1109/74.250128.

[DH01] D. Day and M. A. Heroux. "Solving Complex-Valued Linear Systems via Equivalent Real Formulations." In: *SIAM Journal on Scientific Computing* 23.2 (Jan. 2001), pp. 480–498. ISSN: 1064-8275, 1095-7197. DOI: 10.1137/S1064827500372262.

[DII:GMRES] *The Deal.II Library: SolverGMRES< VectorType > Class Template Reference*. URL: https://dealii.org/9.4.1/doxygen/deal.II/classSolverGMRES.html (visited on 03/25/2026).

[Fog22] A. Fog. "VCL C++ Vector Class Library Manual." In: (Aug. 7, 2022).

[GD09] N. A. Gumerov and R. Duraiswami. "A Broadband Fast Multipole Accelerated Boundary Element Method for the Three Dimensional Helmholtz Equation." In: *The Journal of the Acoustical Society of America* 125.1 (Jan. 1, 2009), pp. 191–205. ISSN: 0001-4966, 1520-8524. DOI: 10.1121/1.3021297.

[GR87] L. Greengard and V. Rokhlin. "A Fast Algorithm for Particle Simulations." In: *Journal of Computational Physics* 73.2 (Dec. 1987), pp. 325–348. ISSN: 00219991. DOI: 10.1016/0021-9991(87)90140-9.

[Kir98] S. Kirkup. *The Boundary Element Method in Acoustics: A Development in Fortran*. Integral Equation Methods in Engineering 1. Hebden Bridge: Integrated Sound Software, 1998. 136 pp. ISBN: 978-0-9534031-0-3.

[Klo20] K. Kloberdanz. *OpenMP - Static or Dynamic Scheduling?* May 10, 2020. URL: https://kkloberdanz.github.io/2020-05-10.html (visited on 03/24/2026).

[Koh+21] B. Kohnke, C. Kutzner, A. Beckmann, G. Lube, I. Kabadshow, H. Dachsel, and H. Grubmüller. "A CUDA Fast Multipole Method with Highly Efficient M2L Far Field Evaluation." In: *The International Journal of High Performance Computing Applications* 35.1 (Jan. 2021), pp. 97–117. ISSN: 1094-3420, 1741-2846. DOI: 10.1177/1094342020964857.

[Lam94] L. Lamport. *LaTeX : A Documentation Preparation System User's Guide and Reference Manual*. Addison-Wesley Professional, 1994.

[Liu09] Y. Liu. *Fast Multipole Boundary Element Method: THEORY AND APPLICATIONS IN ENGINEERING*. CAMBRIDGE UNIVERSITY PRESS, 2009.

[Mar08] S. Marburg. "Boundary Element Method for Time-Harmonic Acoustic Problems." In: *Computational Acoustics of Noise Propagation in Fluids*. Ed. by S. Marburg and B. Nolte. Berlin, Heidelberg: Springer, 2008.

[MB15] D. Malhotra and G. Biros. "PVFMM: A Parallel Kernel Independent FMM for Particle and Volume Potentials." In: *Communications in Computational Physics* 18.3 (Sept. 2015), pp. 808–830. ISSN: 1815-2406, 1991-7120. DOI: 10.4208/cicp.020215.150515sw.

[Rok90] V. Rokhlin. "Rapid Solution of Integral Equations of Scattering Theory in Two Dimensions." In: *Journal of Computational Physics* 86.2 (Feb. 1990), pp. 414–439. ISSN: 00219991. DOI: 10.1016/0021-9991(90)90107-C.

[SB] J. J. Shirron and I. Babuska. "A Comparison of Approximate Boundary Conditions and Infinite Methods for Exterior Helmholtz Problems." In: ().

[SS86] Y. Saad and M. H. Schultz. "GMRES: A Generalized Minimal Residual Algorithm for Solving Nonsymmetric Linear Systems." In: *SIAM Journal on Scientific and Statistical Computing* 7.3 (July 1986), pp. 856–869. ISSN: 0196-5204, 2168-3417. DOI: 10.1137/0907058.

[Wu00] T. Wu. *Boundary Element Acoustics: Fundamentals and Computer Codes*. Southampton, UK: WIT Press, 2000.

[XWZ23] C. Xie, H. Wu, and J. Zhou. "Vectorization Programming Based on HR DSP Using SIMD." In: *Electronics* 12.13 (July 3, 2023), p. 2922. ISSN: 2079-9292. DOI: 10.3390/electronics12132922.

A. Appendix: Code

A.1. Morton Encoding

```
// Expands a 21 bit integer into 63 bits by inserting 2 zeros between each bit
inline uint64_t expand_bits_3d(uint32_t v) {
    uint64_t x = v & 0x1FFFFF;
    x = (x | (x << 32)) & 0x1F00000000FFFF;
    x = (x | (x << 16)) & 0x1F0000FF0000FF;
    x = (x | (x << 8)) & 0x100F00F00F00F00F;
    x = (x | (x << 4)) & 0x10c30c30c30c30c3;
    x = (x | (x << 2)) & 0x1249249249249249;
    return x;
}
inline uint64_t morton_code_3d(uint32_t x, uint32_t y, uint32_t z) {
    return expand_bits_3d(x) | (expand_bits_3d(y) << 1) | (expand_bits_3d(z) << 2);
}
```

Listing A.1: Morton Encoding for 3D

A.2. Real-Equivalent System

```
struct RealFMMOperator : public Subscriptor
    {
        const FMMBEMOperator<dim - 1, dim> &complex_op;
        mutable Vector<std::complex<double>> c_src;
        mutable Vector<std::complex<double>> c_dst;

        RealFMMOperator(const FMMBEMOperator<dim - 1, dim> &op)
            : complex_op(op) {}

        void vmult(Vector<double> &dst, const Vector<double> &src) const
        {
            const unsigned int n_dofs = src.size() / 2;

            if (c_src.size() != n_dofs)
            {
                c_src.reinit(n_dofs);
                c_dst.reinit(n_dofs);
            }

            for (unsigned int i = 0; i < n_dofs; ++i)
            {
                c_src[i] = std::complex<double>(src[i], src[i + n_dofs]);
            }

            complex_op.vmult(c_dst, c_src);

            for (unsigned int i = 0; i < n_dofs; ++i)
            {
                dst[i] = c_dst[i].real();
                dst[i + n_dofs] = c_dst[i].imag();
            }
        }
    } real_fmm_operator(fmm_operator);
```

Listing A.2: Conversion to Real-Equivalent System

A.3. Preconditioning for GMRES

```
class RealDiagonalPreconditioner : public Subscriptor
{
private:
    unsigned int N;
    std::vector<std::complex<double>> inv_diag;

public:
    RealDiagonalPreconditioner(const SparseMatrix<std::complex<double>> &mat)
    {
        N = mat.m();
        inv_diag.resize(N);
        for (unsigned int i = 0; i < N; ++i)
        {
            // Extract the exact diagonal element from our near-field matrix
            std::complex<double> diag_val = mat(i, i);

            // Prevent division by zero
            if (std::abs(diag_val) < 1e-14)
            {
                inv_diag[i] = 1.0;
            }
            else
            {
                inv_diag[i] = 1.0 / diag_val;
            }
        }
    }

    void vmult(Vector<double> &dst, const Vector<double> &src) const
    {
        // Apply the inverse diagonal block to the 2N vector
        for (unsigned int i = 0; i < N; ++i)
        {
            std::complex<double> val(src[i], src[i + N]);
            std::complex<double> precond_val = val * inv_diag[i];
```

```
            dst[i] = precond_val.real();
            dst[i + N] = precond_val.imag();
        }
    }
};
```

Listing A.3: Preconditioning for GMRES